

Cryptography - 11_11

Today we start talking about Cryptomania.

So far, we've seen that if OWF exists, then

$$OWF \implies PRFs \implies PRF \xRightarrow[SKE]{(CPA/CCA)} PRP.$$

Today, we start exploring another world, that is Cryptomania.

We'll study PKE, Public Key Encryption, with CPA/CCA security, and CRH, Collision Resistant Hash Functions.

People have proved that $OWF \not\Rightarrow CRH$, as well as $OWF \not\Rightarrow PKE$.

We have, however, that $TDP \implies PKE$ (Trap Door Permutation).

And we have $KE \iff PKE$ (Key Exchange).

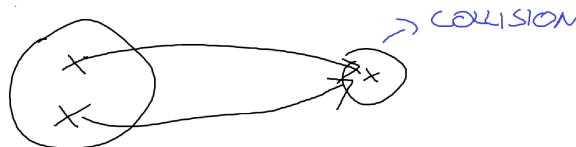
In the world of Minicrypt we also have Digital Signatures (DS), such that $OWF \implies DS$.

Let's start with Collision-Resistant Hash Functions.

This is about families of functions

$$\mathcal{H} = \{h_s : \{0, 1\}^l \rightarrow \{0, 1\}^n\}_{s \in \{0, 1\}^\lambda}$$

such that $l = l(n) \gg n$.



Recall: When we studied PRFs we've seen the construction $\mathcal{F}(\mathcal{H})$, which is a way to extend the domain of any PRF $\mathcal{F}$.

When the seed is public, CRH are only possible for computationally bounded attackers.

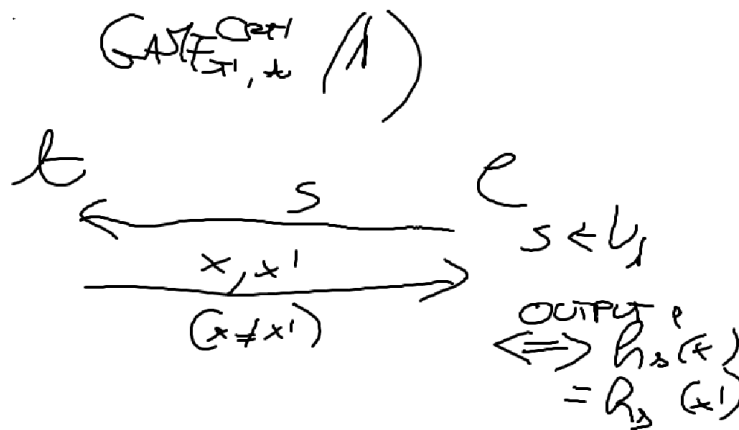
Otherwise he could try all the possible inputs to find a collision.

There are many real world examples: MD5, SHA1, SHA2, SHA3, Merkle Trees, ...

✓ Definition

We say that $\mathcal{H}$ is collision resistant if $\forall PPT \mathcal{A}$:

$$\Pr[GAME_{\mathcal{H}, \mathcal{A}}^{CRH}(\lambda) = 1] \leq \text{negl}(\lambda)$$

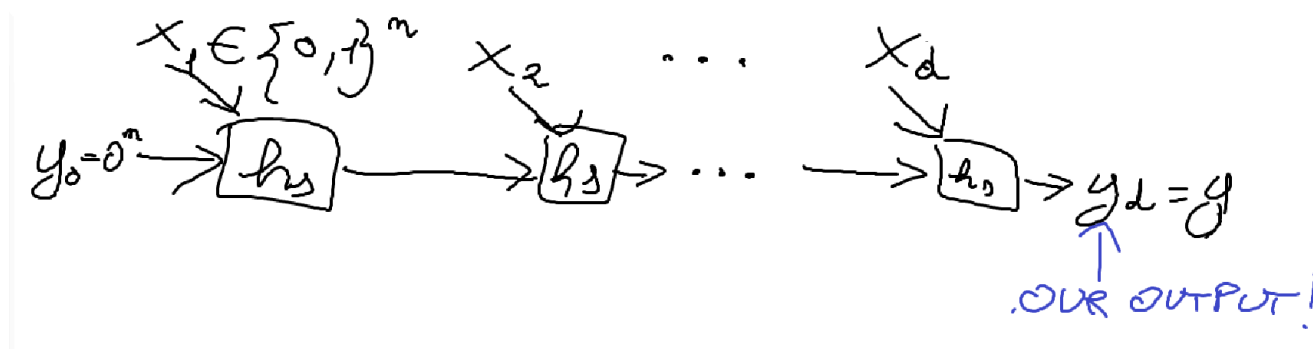


Typical application: Hash a long message and Sign it/MAC it/PRF it.

An important remark: Why do we need a seed? In fact, SHA for instance does not have any seed! We can't rule out the attacker $\mathcal{A}_{x,x'}$ which has a collision x, x' hard-wired and outputs it.

First construction: Merkle-Damgaard transformation. This is behind MD5, SHA1, SHA2. The idea is to obtain CRH H with domain $\{0, 1\}^*$ assuming CRH

$h_s : \{0, 1\}^{2n} \rightarrow \{0, 1\}^n$.



So we compress each block with h_s and y_{i-1} , with $y_0 = 0^n$.

✓ Theorem

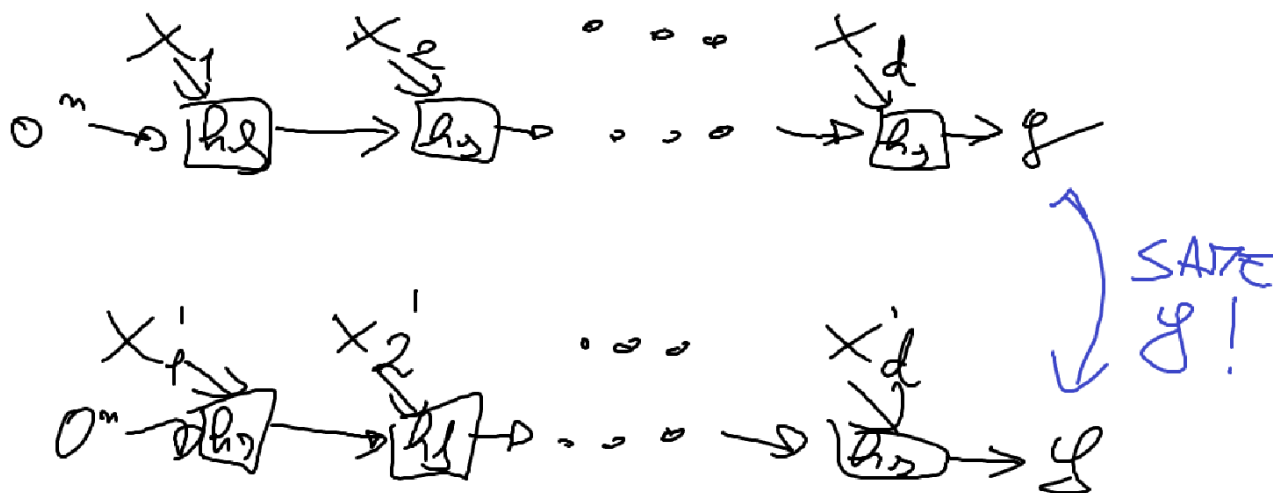
Assuming $\{h_s : \{0, 1\}^{2n} \rightarrow \{0, 1\}^n\}$ is Collision Resistant (CR), then $\{H_s : \{0, 1\}^{d \cdot n} \rightarrow \{0, 1\}^n\}$ is also CR for every fixed $d \in \mathbb{N}$.

① Proof

We basically observe that a collision $x, x' \in \{0, 1\}^{n \cdot d}$ ($x \neq x'$) for H_s implies a collision for h_s . Moreover, the latter collision is efficiently computable.

This immediately implies a reduction, so no adversary can find a collision for H_s .

Let $x = (x_1, \dots, x_d), \neq x' = (x'_1, \dots, x'_d)$ such that $H_s(x) = H_s(x')$.



In order to find the collision for h , we take the largest index among the chain where the inputs are different, but the output is the same.

Looking backwards (from right to left), let $j \in [d]$ be the largest index such that $h_s(x_j, y_j) = h_s(x'_j, y'_j)$ and $(x_j, y_j) \neq (x'_j, y'_j)$. Such j always exists because $x \neq x'$. Now, (x_j, y_j) and (x'_j, y'_j) are a collision for h_s .

■

Unfortunately, this does not work for $\{0, 1\}^*$. This is because we can't rule out that $h_s(0^{2^n}) = 0^n$ while $\{h_s\}$ is still CR. If this is true, then for any x :

$$H_s(x) = H_s(0^n || x) = H_s(0^{2^n} || x)$$

To fix this, we count the number of blocks, and we add one more. The new block will contain the length. Then, the example shown above does not hold anymore, it's not a collision as the length is different, so will be the result.

To avoid it: Encode x such that no legal encoded x can be a suffix of another input.



$\langle d \rangle$: # of blocks encoded using n bits.

Note: you can only hash inputs of at most 2^n blocks, but this is huge for real values of n (e.g. $n = 256$).

✓ Theorem

Assuming $\{h_s\}$ is as before, then the modified $\{H_s\}$ as above is CR for $\{0,1\}^*$ (arbitrary input length).

Proof

We follow the same strategy.

Let $x = (x_1, \dots, x_d)$ and $x' = (x'_1, \dots, x'_{d'})$ be a collision for H_s .

There are 2 cases:

- $d \neq d'$ ($|x| \neq |x'|$). Then $(\langle d \rangle, y_d) \neq (\langle d' \rangle, y'_{d'})$ but $h_s(\langle d \rangle, y_d) = h_s(\langle d' \rangle, y'_{d'})$. This is a collision, as we are assuming y 's are the same.
- $d = d'$. As before.

■

How to build $h_s : \{0,1\}^{2n} \rightarrow \{0,1\}^n$? In practice: heuristically (MD5, SHA1, SHA2).

In theory, either you use number-theoretic assumptions (factoring, discrete log, post-quantum assumptions).

Middle ground solution: Use AES.

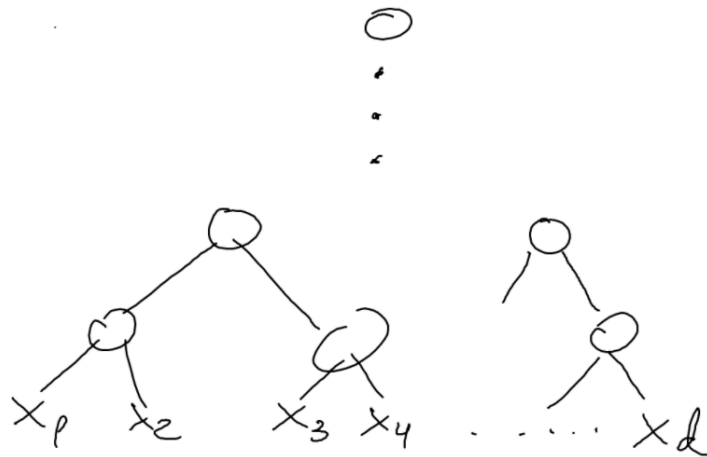
$$h_s(x_1, x_2) = AES(x_1, x_2) \oplus x_2$$

Caveat: We can prove it secure only assuming AES in an ideal cipher, which is a truly random permutation for every choice of the key.

A couple of more facts about hash functions:

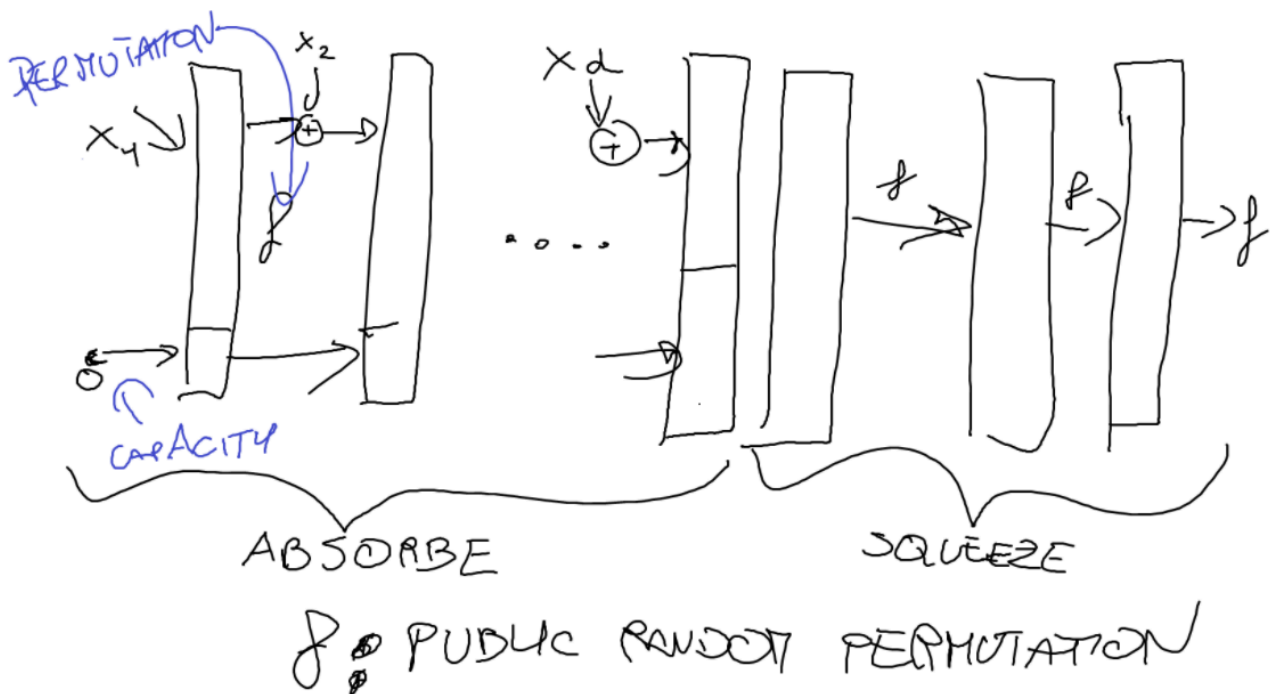
1. Alternative constructions.

One is the Merkle tree.



The top node is the output y .

We also have SHA-3. It's a sponge construction.



Another application of hash functions is to build MACs: The standard HMAC.

Based on the idea:

$$\text{Tag}(k, m) = H(k || m)$$

If $H(\cdot)$ is a random function (random oracle model) this is ok. Not secure if $H(\cdot)$ is built from Merkle Damgaard.

Simple exercise: length extension attack.

Given $\tau = H_s(k || m)$ we can forge on $m^* = (m_1 || \dots || m_d || m')$ by outputting $\tau^* = h_s(\tau || m') = H_s(k || m^*)$.

It can be adapted to the case with the suffix-free encoding.

$$HMAC((k_1, k_2), m) = H_s(k_2, H_s(k_1 || m))$$

k_1, k_2 derived from same k .

Now, let's take a look at number theory.

We will introduce some concrete assumptions:

- Factoring
- Discrete log
- Learning with errors

Number theory is about modular arithmetic $(\text{mod } n)$, namely

$$\mathbb{Z}_n = \{0, 1, 2, \dots, n-1\}$$

Then you can have structures like $(\mathbb{Z}_n, +)$, $(\mathbb{Z}_n, +, \cdot)$, with $+$ and $\cdot \pmod n$.

For instance $(\mathbb{Z}_n, +)$ is a group.

The situation is different for $(\mathbb{Z}_n, \cdot)$, it is not always a group.

Lemma

If $\gcd(a, n) > 1$, then $a \in \mathbb{Z}_n$ not invertible $\pmod n$ with respect to the product $(\cdot)$.